information

MDPI

Review

Countermeasures Against Fault Injection Attacks in Processors: A Review

Roua Boulifa *, Giorgio Di Natale * and Paolo Maistri *

Univ. Grenoble Alpes, CNRS, Grenoble INP1, TIMA, 38000 Grenoble, France

* Correspondence: roua.boulifa@univ-grenoble-alpes.fr (R.B.); giorgio.di-natale@univ-grenoble-alpes.fr (G.D.N.); paolo.maistri@univ-grenoble-alpes.fr (P.M.)

Abstract: Physical attacks pose a significant threat to the security of embedded processors, which have become an integral part of our daily lives. Processors can be vulnerable to fault injection attacks that threaten their normal and secure behavior. Such attacks can lead to serious malfunctions in applications, compromising their security and correct behavior. Therefore, it is crucial for designers and manufacturers to consider these threats while developing embedded processors. These attacks may require only a moderate level of knowledge to execute and can compromise the normal behavior of the targeted devices. These attacks can be faced by developing effective countermeasures. This paper explores the main existing countermeasures against fault injection attacks in embedded processors, to understand and implement effective solutions against those threats. Subsequently, we further investigate solutions related to RISC-V, focusing on its hardware and architecture security.

Keywords: hardware security; fault injection attacks; countermeasures; RISC-V security

1. Introduction

Nowadays, we use digital systems in every aspect of our lives. Energy, transportation, communications, and production tools integrate a significant amount of technology and are increasingly interconnected. This growing importance of digital systems makes them ideal targets for malicious actors seeking to profit or generate economic or political or military destabilization. Therefore, the security of these systems is a crucial issue for the proper functioning of our society.

In recent years, we have become aware of a significant number of different types of weaknesses and attacks. Words such as virus, malware, ransomware, and Trojan horses are now well known to the general public. These attacks generally fall under the category of software attacks, aiming to exploit a lack of users' competences and bugs in programs or protocols. However, attacks can also take other, less publicized forms. A digital system is composed of software applications that are executed on a hardware architecture, which itself can be a direct target of physical attacks.

Physical attacks are generally classified as passive or active [1]. In passive attacks, the targeted device operates normally, and the attacker's goal is to remain undetected while correlating system activity with observable physical parameters such as computation time, power consumption, electromagnetic emissions. Among these attacks, notable examples are Differential Power Analysis [2] and Correlation Power Analysis [3] (or their EM counterparts), which allow effective secret recovery with a few hundredths to thousands of traces, depending on the degree of implemented countermeasures. Active attacks, in contrast, involve the adversary tampering with the target device by manipulating operational or

check for updates

Academic Editor: Christoforos Kachris

Received: 24 January 2025
Revised: 9 March 2025
Accepted: 14 March 2025
Published: 5 April 2025

Citation: Boulifa, R.; Di Natale, G.; Maistri, P. Countermeasures Against Fault Injection Attacks in Processors: A Review. *Information* **2025**, *16*, 293. https://doi.org/10.3390/info16040293

environmental conditions such as input voltage or clock, electromagnetic perturbations (laser, EM, ...), or temperature, leading to faulty behavior and disruptions. Altering the normal behavior of a device during execution by inducing one or more faults may be further exploited as a vulnerability.

Fault injection attacks (FIAs) [4] are a well-known type of active attack studied extensively over the past two decades, and they are typically accomplished by manipulating environmental parameters or inputs. With a small number of experiments, FIAs can reveal secret data and help intruders achieve various objectives, including causing erroneous outputs, disturbing normal behavior, bypassing specific operations, and extracting sensitive information from faulty outputs. Although the scope of fault injection attacks can be usually limited by the need for physical access to the targeted circuit, it would be a mistake to neglect this aspect because the overall level of information system security is defined by the security level of the weakest link. Several countermeasures have been proposed to protect against fault injection attacks [5], for example, by securing cryptographic algorithms, protecting against reverse engineering, or implementing special security measures at design time [6].

In this work, we review the most important countermeasures against fault injection attacks targeting processors, with a special focus on the RISC-V architectures. By comparing these solutions, we highlight the intrinsic characteristics and properties of countermeasures, which can be helpful for security engineers to develop secure processors.

Other works have analyzed the physical security of embedded systems. For instance, refs. [7,8] examine hardware security threats and mitigation strategies, but they do not comprehensively address fault injection attacks across all levels (electrical logic, ISA, microarchitecture, and software). Unlike previous reviews, this study systematically examines fault injection attacks across multiple abstraction levels, focusing specifically on countermeasures applicable to processor architectures. A more detailed overview of previous surveys and their respective differences from our work is reported in the next section.

This paper is organized as follows: Section 2 describes fault injection techniques used for physical attacks. Section 3 presents the corresponding fault models at different abstraction levels. Section 4 reviews the state of the existing counter measures and compares them by using different metrics. Section 5 outlines the characteristics of an efficient counter measure that can be implemented in RISC-V. Finally, Section 6 presents the conclusion of this work.

Related Works

Other research works have reviewed the physical security of embedded systems from different points of view. Mahmoud et al. [9] provides a broad analysis of security threats targeting modern computing systems, with a particular focus on fault injection attacks in hardware to manipulate software execution. It covers various fault injection techniques, including side-channel attacks, voltage glitching, and cryptographic security mechanisms. The study examines hardware and software-based detection methods, including machine learning and dynamic code analysis. However, the study primarily focuses on IoT devices rather than general-purpose processors. Shuvo et al. [8] specifically focuses on non-invasive fault injection techniques such as clock glitching, voltage manipulation, and electromagnetic fault injection (EMFI). It highlights the increasing accessibility of these techniques and their impact on processors and embedded systems. While it provides valuable insights into the evolution of non-invasive fault attacks, it does not cover invasive techniques, nor does it provide a structured discussion on mitigation strategies at different levels of abstraction. Mishra et al. [7] offers a comprehensive overview of hardware security threats, including fault injection attacks. However, its primary focus is on general hardware security

concerns, covering a broad spectrum of attacks beyond fault injection. While it discusses various mitigation strategies, some of them are specific to RISC-V architecture, it does not systematically examine fault injection countermeasures across different levels of abstraction (electrical, logic, ISA, microarchitecture, and software).

Kazemi et al. [10] provides a foundational review of fault injection attacks on cryptographic systems, classifying attacks based on cost, complexity, and effectiveness. While it provides an in-depth analysis of these specific attacks, it does not extend its discussion to fault injection countermeasures.

Barenghi et al. [11] examines fault injection techniques in the context of microcontroller-based IoT applications, specifically exploring clock glitching and voltage fault injection. It highlights existing evaluation platforms that facilitate security testing against such attacks, emphasizing the need for open-source tools to assess the resilience of embedded devices. Although it provides an in-depth analysis of these specific attacks, it does not extend its discussion to mitigation techniques, such as architectural and software-based mitigation strategies.

A few works [12–14] have reviewed attacks leveraging microarchitectural vulnerabilities: they mostly focus on covert and/or side channels, where timing attacks are carried out exploiting insecure implementations of caches, speculative execution, etc. Fault injection attacks, on the other hand, are not taken into consideration.

Unlike previous review papers, our study systematically examines fault injection attacks across multiple abstraction levels, providing a detailed evaluation of countermeasures applicable to processor architectures. By analyzing mitigation strategies at various layers from electrical faults to software-based protections. Our paper offers a unique perspective that bridges the gap between attack methodologies and practical countermeasure implementations. Additionally, our review emphasizes the challenges associated with real-world adoption, including performance overheads, portability, and implementation constraints.

2. Fault Injection Attacks

Fault injection attacks (FIAs), or fault attacks, are a type of active attack where the goal of the attacker is to disrupt the circuit functionality during its operation. The issue of faults was first known in the aerospace industry, where systems, less protected by our atmosphere, can be disturbed by particles' impacts. However, Boneh et al. [4] were the first to investigate the consequences of errors in the security field, by intentionally generating faults to break the security of digital systems. Fault injection mechanisms allow an attacker to disturb the system physically, the goal being to induce modifications in the system, by generating abnormal states, which can be exploited to retrieve valuable assets (for instance, secret keys or privilege escalation). Fault attacks are generally non-destructive, as their goal is not to permanently degrade the circuit but to momentarily disrupt it.

Fault attacks have many parameters to be controlled, such as spatial precision, i.e., the ability to precisely select one or more bits; the effect of the fault on these bits (inversion, setting to 1, setting to 0, random transformation); their temporal precision, i.e., the ability to control the moment of injection, for example, during the execution of an instruction; and their permanence, i.e., the duration of the perturbation or its effect over time (one cycle, several cycles, or permanent).

When constructing an attack, the fault injection mechanism is chosen based on these criteria and the intended goal. Some attacks can be carried out with a low level of precision, while others require maximum precision to achieve the desired goal or to avoid triggering a countermeasure.

In the upcoming subsections, we will present various techniques of fault injection attacks, each with its own characteristics. The covered techniques are clock glitching,

electromagnetic (EM) fault injection, voltage-based attacks, laser-induced faults, and X-ray-based attacks. In Table 1, we compare the different techniques of physical attacks with respect to three distinct criteria: temporal precision, spatial precision, and cost.

Table 1. Physical attack precision comparison.

Physical Attack	Temporal Precision	Spatial Precision	Cost
Clock glitch	High	N/A	Low
EM injection	High	Medium	Medium
Voltage glitch	High	N/A	Low
Laser	Very High	High	High
X-ray	N/A	Very High	Very High
Software	Medium/High	N/A	Very Low

2.1. Clock Glitching

Integrated circuits use the clock to synchronize the sequential elements (i.e., flip-flops). On clock edges (rising and/or falling), synchronous elements change their state based on the value of their inputs. The minimum clock period of a circuit is defined by the maximum time required for a signal to propagate through the combinational logic between two synchronous elements. Below this value, it is possible that the input signals of synchronous elements are not stable and an erroneous value may be sampled.

The goal of a clock glitch is to shorten the period of one (or multiple) clock cycles, so that erroneous inputs are sampled by the flip-flops. This attack is possible when the clock can be easily manipulated by the attacker.

Agoyan et al. [15], use glitches on the clock signal to disrupt the calculation of the AES algorithm on Xilinx Spartan 3AN and then perform a cryptanalysis attack on the erroneous result. They notably show the relationship between the duration of the clock signal disruption and the number of faulty bits in the result.

Claudepierre et al. [16] propose a low-cost mechanism for injecting multiple faults using clock signal perturbations.There are also low-cost fault injection systems marketed for the general public, such as the ChipWhisperer [17] boards which allow to control glitches with an advanced triggering mechanism and a clock glitch generator.

Nevertheless, clock signal perturbation mechanisms usually allow for low spatial precision. Indeed, it is not possible to control the value of the fault, but it is possible to fault critical paths and, in some cases, to inject single bit errors [15]. On the other hand, the temporal precision is limited to the clock period in which the perturbation is injected. Such good temporal precision may reflect in spatial precision as well when targeting microcontrollers or CPUs: in this case, the sequential execution of algorithm's code allows to finely target specific instructions, hence specific registers or program's behaviors.

This type of perturbation is non-permanent and the circuit returns to its nominal operation in the following cycle. However, the effect of the perturbation can propagate over several cycles, for example, when the perturbation modifies the value contained in a register.

2.2. Voltage Glitching

Digital devices are designed to operate with a stable power source. In this circumstance, transistors, the main components of digital circuits or devices, tend to operate in the cut-off and saturation regions (OFF/ON) to indicate logic values of 1 and 0. Modifying the supply voltage can have effects on the normal behavior of the transistor.

The goal of voltage glitching is to manipulate the power supply, causing faulty behavior on a device. It can be achieved by creating high variations in a power supply or by under-powering the device.

Korak et al. [18] combine glitches on the clock signal with glitches on the power supply bus to improve the success rate of fault injection. They report that the duration of the glitch allows targeting different stages in the micro-architecture of the target micro-controllers. Voltage glitching attacks were even used to break security enclaves of Intel [19] and AMD [20].

As for clock glitches, the perturbation is non-permanent, with the circuit returning to normal operation in the next cycle. However, the effect can propagate across multiple cycles.

2.3. Electromagnetic Injection

Electromagnetic fault injection (EMFI) is another mechanism to perturb the circuits by generating an electromagnetic field close to the device, which allows inducing currents within the design nets, leading to faults.

According to the model proposed by Dumont et al. [21], during an electromagnetic pulse, the power network of the circuit is disturbed for the duration of the pulse. This disturbance particularly affects the synchronous elements of the circuit. Moreover, they report that the polarization of the electromagnetic pulse allows controlling the nature of the fault (setting to 0 or 1), and the propagation of the disturbance remains localized around the injection probe. They explain and model how EMFI induces faults in integrated circuits by generating two electromagnetic pulses with opposite polarities. The first pulse induces a transient reversal of the supply voltage, while the second pulse restores it. During the supply voltage recovery phase, digital flip-flops can sample a wrong value because they operate under degraded operating conditions.

Ordas et al. [22] demonstrated that EM injection can produce not only timing faults when the EM injection interferes with the signals within the circuit, causing delays or unexpected changes in the timing of operations but also bit set and bit reset faults. This technique achieves good spatial and temporal precision with a localized effect with the proper equipment. However, the complexity of implementation and the cost of the hardware are higher than those of the voltage and clock perturbation mechanisms.

2.4. Laser Injection

Laser fault injection (LFI) was first described in the context of hardware security by Skorobogatov and Anderson [23]. The goal of LFI is to inject faults into integrated circuits using the photoelectric effect, which results from the interaction between the laser and silicon. The flow of the current pulse generates a transient voltage pulse within the circuit's logic. This pulse has the potential to propagate through the circuit, leading to a fault in its operations. Alternatively, laser-induced faults can directly impact memory elements such as RAM or registers.

LFI offers distinct advantages: the laser enables an attacker to inject faults into a very specific point of interest in the target device with very high precision in both spatial and temporal dimensions.

2.5. X-Ray Injection

X-ray modification is a powerful semi-permanent fault injection technique with a high spatial accuracy, which allows an adversary to modify efficiently secret data from an electronic device. It is physically possible to modify the state of a single transistor with the X-rays and the limitation is only coming from the way used to focus the beam. X-rays can penetrate deeply through materials and induce semi-permanent faults on flash memory cells and NMOS transistors.

In 2017, Anceau et al. proposed an approach to modify the behavior of a transistor in the memory of a circuit using focused X-ray beams [24]. The perturbation is semi-permanent in this case, and it is necessary to heat the circuit to make it disappear. This is in contrast to the injection mechanisms presented earlier, for which the perturbation disappears by itself after a few clock cycles. This type of attack opens up new perspectives but is still limited by the cost of accessing the necessary equipment.

2.6. Software Based Fault Attack

In the previously mentioned mechanisms, fault injection traditionally required physical access to the device; however, faults can also be triggered remotely by a virtual attacker using malicious software. This shift means that faults are no longer purely environmentally dependent but instead arise from internal software-controlled mechanisms.

As explained by Kim Yoongu et al. [25], Rowhammer is a well-known attack that exploits the electrical properties of DRAM by rapidly and repeatedly accessing a specific row of memory cells, leading to bit flips in adjacent rows. This software-induced memory corruption can enable privilege escalation and cryptographic key extraction without requiring physical access to the hardware.

Similarly, Plundervolt [26] is another software-based fault injection attack that manipulates power management mechanisms to induce computation errors inside security-critical environments, such as Intel SGX enclaves. Murdock et al. discovered that by modifying CPU voltage through dynamic voltage and frequency scaling (DVFS), attackers can introduce controlled faults, compromising cryptographic operations and exposing sensitive data. Ref. [27] provides a broader overview of internal energy-based FIAs, discussing how attacks like Plundervolt exploit voltage scaling mechanisms via malicious kernel modules or drivers. These attacks force the system into unstable operational states by either overclocking (increasing frequency beyond limits) or undervolting (reducing supply voltage while maintaining high frequency), leading to transient computation errors.

This emerging class of energy-based fault attacks poses significant threats, as demonstrated in recent research [27], which highlights FPGA-to-CPU undervolting attacks as another potential vector. While earlier studies focused on physical voltage glitching, these software-driven attacks highlight a shift toward scalable and remote software-controlled fault injection techniques that compromise modern trusted execution environments (TEEs).

In the following, we specifically focus on attacks requiring physical access to the device. Although such attacks can be more challenging to mount, as they necessitate specialized tools, advanced expertise, and direct proximity to the target, they also represent vulnerabilities that are significantly harder to mitigate. Therefore, software-based fault attacks discussed in this subsection, remain outside the scope of our analysis.

3. Fault Modeling

Fault models describe the effect of physical injection and are used in simulation environments to predict how the system behaves under fault conditions before testing on real hardware. Simulations can be carried out at different levels of abstraction: the closer the simulation is to the physical hardware, the more accurate it becomes, but at the cost of longer simulation times.

The impact of a fault injection depends on the specific attack scenario and the targeted system or component. Different types of attacks may exhibit distinct fault effects based on their objectives and the vulnerabilities they exploit. Therefore, fault models depend on the type of physical injection. Moreover, fault models can be defined at different abstraction levels, presenting trade-off between accuracy and simulation time.

When studying fault effects, it is crucial to consider the specific context, attack vectors, and targeted systems to accurately assess the potential consequences and devise appropriate countermeasures. The nature of the fault, the affected components, and the system's response to the injected fault all contribute to the observed effects.

Table 2 summarizes the fault models at the different abstraction levels.

3.1. Electrical and Logic Level

The following paragraph presents models at both the electrical and logic levels in the case of fault injection attacks. Most physical attacks typically induce a glitch in voltage or current at a specific point in the circuit. The voltage or current injection fault model can be implemented by adding a voltage or current source connected to the affected point to simulate the effect of the attack. The glitch can be modeled as a simple rectangular pulse or, for more precision, as a double exponential, which is a waveform characterized by two exponential terms, accurately depicting the rise and fall of the glitch. For example, a single event transient (SET) is a voltage pulse striking the blocked junction of a MOSFET transistor. The resulting transient current pulse (SET) may propagate through logic gates, potentially causing data corruption or system failure.

At the logic level of abstraction, a system is represented as a network of interconnected logic gates, which perform fundamental operations and serve as the building blocks of digital circuits. Simulators operating at this level model the behavior of individual gates and their interconnections, allowing the representation of larger components, such as multiplexers, registers, or ALUs.

Fault injection at the logic level involves simulating how faults affect the operation of these gates or the signals passing through them. They can grouped into two categories: transient and permanent faults.

Transient faults cause temporary disruptions in logic values, which can be corrected when the affected values are updated or overwritten by new inputs. In this category, single-event upset (SEU) results in a bit flip of the logical state of a memory element. SEUs may model ionizing particles depositing charges on memory points, or through the transformation of transient upsets into logical errors. Bit-level errors such as bit set (where a logic bit is forced to 1) and bit reset (where a logic bit is forced to 0) are other examples of transient faults that disrupt the normal operation of the circuit. In addition to SEUs, multiple bit upsets (MBUs) can occur, where multiple bits are affected simultaneously.

In contrast, permanent faults result in lasting effect to the circuit's functionality. A common type of permanent fault is the stuck-at fault, where a gate input or output is stuck at a fixed logic value (either 0 or 1), irrespective of the intended operation. These faults can model physical issues such as shorts to ground or Vdd, or permanent damage caused by laser injections. Other permanent faults include delay faults, where gate delays are altered, resulting in timing violations, and transition faults, where incorrect transitions occur at the outputs of gates.

3.2. Microarchitectural Level

The characterization works aim to build fault models at the microarchitecture level [28]; faults are typically considered as impacting only a single Micro-Architectural Block, as demonstrated by T. Trouchkine et al. in their study [23]. These faults can manifest in different ways, affecting either the pipeline, registers [29], or memory. They can be classified into two main categories: those that influence data, and those that impact instructions. Under the data-related category, we may encounter issues like register or memory corruption, as well as bad fetch. On the other hand, faults under the instruction-related category can result in corruption within the pipeline micro-architectural blocks (MABs), cache, or instruction

bus. One case of a fault is "bad fetch", which can arise from either loading the wrong instruction into the instruction cache or encountering a failure in address translation.

In [30], the authors noticed, using only RTL fault simulation on a RISC-V core, that bit flips could lead to forwarding faults, leading in certain scenarios to breaking the control flow integrity of programs.

3.3. ISA Level

At the instruction set architectural level fault injection leads to affect the execution of the program. As a result, they proposed various fault models, including

faulty computations, data corruption, or unexpected behavior during program execution which can disrupt the control flow. This disruption may alter the program counter (PC) values, causing the execution to jump to unintended locations or skip crucial instructions. Skipping instructions can affect one [31], two [32] or multiple [33] instructions and it is one of the most known fault models at the ISA (instruction set architecture) level. Recently, A. Elmohr et al. [34] showed for the first time that EM fault injection can be injected in a 320 MHz RISC-V processor leading to multiple instruction skip. As a result, if the EM pulse voltage increases, the number of consecutive instructions being skipped also increases. In this case, the duplication and triplication countermeasures are not sufficient: Laser FI has been shown to produce multi-instruction skips as well [35,36].

Fault injection can also lead to instruction corruption [37,38], which can affect the operands or the opcode. In some experiments, authors could not explain the obtained faulty behaviors, such as the corrupted values found in some registers, independently of their actual use in the code. Alshaer et al. [39] have improved the skip (and skip and repeat) fault model by considering the cache buffer, which helped to explain a wide range of the obtained faulty behaviors at the ISA level. Their modeling is independent of the targeted instructions and explains many cases of generic corruption.

In [40], the authors proposed a complementary fault model called partial update, which explains how instruction corruption can be explained due to racing conditions in the path from memory to pipeline stages. Depending on glitch parameters, only a subset of the instruction opcode may be updated correctly, and others will be updated in a faulty way. The corrupted part will either depend on the previous value in the instruction register (IR), or from a precharge value from the bus. In [41], the authors have shown that in a RISC-V target device, a fault could cause an instruction to be skipped with its result being nonetheless forwarded to a later instruction. This happens when the instruction completes the execute stage but cannot write back to the register file, while the correct value is correctly propagated through the forwarding path.

Previous works have focused on ISA fault models such as *instruction skip* (and possibly *repeat*) or corruption; further analysis has shown that the microarchitecture plays a major role, in particular in the fetch stage, by proposing more refined models, such as, for instance, the *partial update*.

3.4. Software Level

At the software level, hardware faults are translated into the software domain, and faults at lower levels are more accurate but they require longer simulations. Faults at this level are particularly significant because they can manifest as complex application errors, algorithmic failures, or disruptions in system-level operations.

We can group those models into three categories: data fault models, code fault models, and system fault models. The first enables model faults that corrupt data processed by a software application; the second allows modeling faults that corrupt the set of instruc-

tions composing a program; and the third models both timing faults and communication/synchronization faults during the software execution.

A. Bosio et al. [42] introduced three different fault abstractions: the first affecting the portion of the instruction responsible for the encoding of the destination register (Rd), which models the data fault model, the second affecting the instruction opcode and modeling the code fault model, and the third affecting the condition flags (cond) of the instruction, which can result on having an erroneous flag impacting the control flow of the program.

We can conclude that the effects of fault injection can vary depending on the specific error injection mechanism, the location of injection, and the fault propagation characteristics of the circuit. By changing the physical attack, we can have different fault effects.

Table 2. Fault models at different levels.

Fault Level	Fault Model
Electrical and Logic level	SET, SEU, MBU, MET Double exponential Current injection Bit flip, Bit Set, Bit Reset Stuck-At Fault Delay Fault Transition Fault
Micro-architectural level	Register Corruption Memory Corruption Instruction Bus Corruption Bad fetch Pipeline and MAB Corruption
ISA level	Instruction Skip, Skip and Repeat Register Corruption Incorrect Instruction Execution Partial update
Software level	Control Flow Error Variable Corruption

4. Existing Countermeasures: Strategies and Implementation Methodology

In this section, our focus will be on general-purpose processors. We begin by categorizing the key features of existing countermeasures; subsequently, we provide detailed descriptions and comparisons within each category. Moreover, we discuss their applicability and portability to RISC-V processors in Section 5.2. The goal of this chapter is to help hardware security engineers to better understand the existing countermeasures and to possibly provide hints on how to trade-off between security, cost, and performance.

Countermeasures can be categorized into various groups, each aligned with specific objectives. Multiple categories may be employed, including a fault model-based classification, as well as differentiation based on the intended application, whether cryptography or general. In this work, we decided to classify them based on the type of technique used to implement the protection, i.e., (i) countermeasures based on the use of encryption algorithms to protect the executable code, (ii) based on spatial redundancy in the micro-architecture of the processor, and (iii) based on the use of signatures to protect chunks of executed code. Moreover, we will highlight the addressed fault models for each countermeasures through different levels of abstractions in Table 3.

4.1. Countermeasures Based on Encryption

Encryption, a foundational security technique, guarantees the confidentiality of information. Many countermeasures employ encryption to exploit its inherent nonlinearity and thus protect the information. In the following, we will describe a few solutions based on this mechanism.

SOFIA [43] aims at protecting the integrity and the control flow of the executed code by decrypting instruction at run time based on the control flow path. To do that, each instruction in the binary is encrypted by using a block cipher in counter (CTR) mode, at compile time. The instructions are encrypted based on the control flow paths present in a precise control flow graph (CFG) of the whole program and decrypted at run time using a combination of the current program counter and the address of the previously executed instruction. It combines the control flow integrity and the software integrity to ensure both integrity and confidentiality: instructions encrypted in the program memory are protected from readout; a run time message authentications code (MAC) is calculated on the decrypted instructions and compared to the precomputed MAC to verify the integrity. If the verification fails, the processor is reset. The authors have reported that applying SOFIA increased the hardware area of their LEON3 core by 12.9% and slightly reduced the maximum clock frequency.

This solution defends against attacks based on code injection, code reuse, software tampering, and fault attacks on control flow. The original authors considered a fault model capable of non-invasive attacks that target the program flow, such as clock glitching. However, other types of fault attacks that do not target the program flow, such as glitching the ALU or bus of the processor, are not considered part of the attacker model.

Another solution based on encryption is sponge-based control-flow protection for IoT devices (SCFP), proposed by Werner et al. [44]: as with SOFIA, it relies on cryptographic methods to enforce control flow integrity, software integrity, and code secrecy. However, SCFP protects the confidentiality of software IP and the authenticity of its execution in IoT devices to achieve better code size and runtime overheads compared to SOFIA. SCFP encrypts programs at compile time using a sponge-based cipher; decryption is then performed within the CPU, instruction by instruction, just in time during execution. SCFP covers from software attack when the attacker has arbitrary read and write access to the memory due to bugs in the software, to active attack, when global faults are injected into the system through, for example, clock glitching.

The implementation of SCFP aims to safeguard against both illegal memory accesses, due to software bugs, and physical attacks such as direct access due to faults injected into the system. SCFP thus offers protection against fault injection, which results in instruction skipping or skipping and repeating, random faults due to glitches, low number of controlled bit flips due to laser, and a limited number of probed wires.

Hardware-assisted program execution integrity (HAPEI) [45] is another solution based on encryption which use a different approach to program state encoding and encryption for ensuring instruction and control flow integrity. Instead of relying on the program counter (PC) or other sequential identifiers, HAPEI encodes the state by considering the entire history of previously executed instructions. This encoding is designed to be independent of the program counter, thereby guaranteeing the correct execution of instructions and maintaining control flow integrity. The encryption process involves using a hash function with a secret key to encode the initial state, and subsequently, each instruction is encoded based on the prior state. For scenarios involving branching or multiple predecessors, HAPEI proposes a method to ensure program integrity and control flow correctness by encoding the program state as a history of all previously executed instructions, rather than using the current and previous program counters. Each instruction is encoded with

a unique state, updated using a hash chain dependent on all prior instructions. This allows for easy validation of the program state at any point during execution. The method supports branching and cycles in the control flow graph by rebasing the program state with a random value and using polynomial interpolation to map predecessor states to a new state. This ensures that instructions are executed correctly and enables frequent integrity checks. HAPEI covers against code injection, code reuse, and fault injection attacks on instructions.

CONFIDAENT [46] (control flow protection with instruction and data authenticated encryption) is also based on code encryption combined with control flow integrity, ensuring the confidentiality and authenticity of code through authenticated encryption. The novelty in this approach is that it handles indirect jumps where the destination address is only known at runtime, such as when calling a dynamic library. The authors investigated physical attacks that have limited access to the device, meaning the attacker can only manipulate elements that are potentially external to the CPU, such as RAM. These manipulations can modify the code by injecting voltage and clock glitches during execution, consequently resulting in faulty instructions due to incorrect decoding or register corruption, such as the program counter (PC). Based on an authenticated encryption and masking mechanism the confidentiality, the authenticity of the code in memory and the integrity of the control flow are ensured. For each instruction, a mask is calculated based on the previous mask; the authors propose an encryption structure that allows fine-grained data writes and supports efficient control flow integrity (CFI) through the addition of second-level instruction masks. The LLVM compiler has been modified to allow easy encryption of already compiled libraries.

Runtime attestation resilient under memory attack (ATRIUM) [47] utilizes again authenticated encryption and masking mechanisms for code confidentiality and integrity, calculating masks for fine-grained data writes. ATRIUM is a runtime attestation scheme designed for bare-metal embedded systems. It involves a prover and a trusted verifier. The former executes the program, while the latter verifies its integrity. ATRIUM ensures that the program's control flow and executed instructions are correct, even in the presence of advanced runtime attacks. The verifier preprocesses the program to create a control flow graph (CFG) and computes hash values for legal execution paths. During execution, ATRIUM hashes the paths taken by the program and sends these hashes along with a signature to the verifier. The verifier checks these against precomputed values to confirm the program's integrity. This process is optimized to handle loops efficiently by segmenting the CFG and computing separate hashes for different segments, ensuring comprehensive and efficient attestation. For each instruction, a mask is calculated based on the previous mask. The llvm compiler has been modified to allow easy encryption of already compiled libraries. This solution covers code injection attacks, code reuse attacks, hardware fault attacks on instructions, and TOCTOU (time of check time of use) attacks.

EC-CFI (Control-flow integrity via code encryption counteracting fault attacks) [48] encrypts each function with a different key, which is dynamically derived before each control-flow edge. Detection of faults is achieved when decrypting with the wrong key, which occurs when a fault redirects a control-flow edge to another function outside of the call graph.

The solutions presented—SOFIA, SCFP, HAPEI, CONFIDAENT, ATRIUM, and EC-CFI—share common elements in leveraging encryption to enhance control flow integrity, software integrity, and confidentiality of code execution. Each approach employs cryptographic techniques to encrypt instructions and validate their integrity at runtime, effectively defending against a range of attacks including code injection, code reuse, and fault injection. However, they differentiate in their specific implementations and targeted attack vectors.

SOFIA uses a block cipher in CTR mode, focusing on protecting program memory and runtime integrity, while SCFP employs a sponge-based cipher for enhanced performance on IoT devices. HAPEI uniquely encodes program state based on the history of executed instructions, independent of the program counter, providing robust branching support. CONFIDAENT innovates with handling indirect jumps and using authenticated encryption to protect against physical attacks. ATRIUM integrates runtime attestation with control flow verification through a trusted verifier, ensuring program integrity against advanced runtime attacks. EC-CFI dynamically derives keys for each function to counteract fault attacks by detecting incorrect control flow redirection.

The strengths of these solutions include their robust protection mechanisms and diverse methodologies to ensure integrity and confidentiality. However, potential flaws may arise in increased hardware complexity, performance overheads, and limited scope of certain fault models, suggesting a need for comprehensive and balanced designs to address the evolving landscape of security threats.

4.2. Countermeasures Based on Spatial and Timing Redundancy

In the previous section, we explained countermeasures based on encryption. In this section, we will explore countermeasures based on spatial redundancy.

T. Chamelot et al. [49] propose a processor extension and software support to protect the system against fault injection attacks: SCI-FI (control signal, code, and control flow integrity against fault injection attacks). This countermeasure ensures code integrity, control flow integrity, and execution integrity. It implements a redundancy-based mechanism in the control signal integrity (CSI) module, duplicating and checking signals between pipeline stages. The countermeasure protects against two types of faults: those occurring in memory and those affecting the processor. These faults may exert full control over a few bits (less than 8 bits) or alter many bits without any control over the resulting values (random bit-flips).

SCI-FI has two security modules to protect the control logic. The first module is the CCFI (code and control-flow integrity); its main function is to verify the signature at run time and at compilation time. This mechanism is enacted using a signature and update function and signature verification. This module implements the GPSA (generalized path signature analysis) technique [50] by calculating a signature based on the pipeline state, which is a set of control signals independent of the data. The pipeline state accurately represents the binary encoding of instructions to ensure code integrity. Chaining of signatures is used to detect any deviations in the control flow. Therefore, this module ensures the integrity of code, control flow, and control signals from the fetch stage to the decode stage. The second module is the CSI (control signal integrity module), which verifies the control signals. It implements a redundancy mechanism by duplicating the propagation of selected signals between the different pipeline stages to ensure the integrity of signal propagation in the remaining stages of the pipeline. This provides complete coverage of signal integrity in the pipeline state within the processor's microarchitecture. By combining these two modules, SCI-FI provides comprehensive protection against fault injection attacks, covering both the control logic responsible for executing instructions and the propagation of control signals. An extension of the previous work named MAFIA [51] presents particular support for indirect branches, branch prediction, and interrupts. It also gives more details regarding the hardware and software implementations, and it provides an analysis of MAFIA's security.

CCFI-cache (code and control-flow integrity [52] ensures control flow integrity by fetching additional metadata and by verifying the control flow on-the-fly using redundancy-based checks. It verifies code and CFG using intra-procedural CFI, thus protecting branches

and jumps, and inter-procedural CFI, which considers function calls and returns. CCFI-cache covers backward edge (ROP, buffer overflow), forward edge, code, and fault injection.

CFI is ensured by two additional hardware modules: the CCFI-cache fetches the metadata which has been computed at compile-time, containing all control-flow related information. This information is used at runtime by the second module, the CCFI checker, which follows the execution and verifies the control flow on the fly using the metadata. At the end of each basic block (BB), it checks the validity of the target address by comparing it against the precomputed values contained in the metadata, thereby ensuring intra-procedural CFI. Intra-BB consistency is ensured by a watch-dog counter that controls the number of executed instructions before a control transfer. In case of a function call or return, an integrated shadow stack is used to verify inter-procedural CFI. This shadow stack in embedded inside de CCFI-Checker and is not accessible from the main processor. Finally, code and metadata integrity is ensured by a precomputed signature that is compared to a hash value over the executed instructions computed at run-time.

The SecDec countermeasure introduced by Leplus et al. [53] targets physical attacks on embedded processors by using a masking technique that relies on signals generated by the previous instruction to protect the current one. Specifically, a mask is created during one cycle (cycle N) and applied during the decode stage in the following cycle (cycle N + 1). This approach is non-random and depends on the previous instruction. The mask is generated at compile time using post-decoder signals such as the register index, immediate values, multiplexer selectors, and enable signals.

In comparing encryption-based and spatial redundancy-based countermeasures for protecting system integrity against fault injection attacks, several common elements and differentiating features emerge. Both approaches aim to ensure code integrity, control flow integrity, and execution integrity. Encryption-based solutions like SOFIA, SCFP, HAPEI, CONFIDAENT, ATRIUM, and EC-CFI leverage cryptographic methods to encrypt and validate instructions at runtime, offering robust protection against various attacks, though often with increased hardware complexity and potential performance overheads. On the other hand, spatial redundancy-based solutions such as SCI-FI and CCFI-cache focus on duplicating and checking signals or control flow paths to detect and correct faults. SCI-FI uses a combination of control signal integrity (CSI) and code and control-flow integrity (CCFI) modules to protect against memory and processor faults by verifying control signals and pipeline states. CCFI-cache ensures control flow integrity using metadata and run-time verification, covering both intra- and inter-procedural CFI with additional hardware modules like shadow stacks and watchdog counters. The strength of spatial redundancy lies in its comprehensive fault coverage and real-time error detection capabilities, but it may introduce redundancy overhead and complexity in signal management. Both methods have their strengths in enhancing security, yet their flaws highlight a trade-off between complexity, performance, and the scope of attack vectors they address.

Table 3. Fault model coverage across various countermeasures.

Fault Model	SCFP [44]	MAFIA [51]	CFI [54]	SOFIA [43]	CONFI-DAENT [46]	CCFI-Cache [52]	ATRIUM [47]	HAPEI [45]	CIFER [55]	SecDec [53]
SET										
MET										
SEU/Bit flip	✓	✓								✓
MEU										
Bit set		✓								
Bit reset		✓								
Stuck at fault										
Delay fault										
Transition fault										
Register corruption		✓								✓
Memory corruption										✓
Instruction bus corruption		✓								
Pipeline MAB corruption		✓								✓
Instruction skip	✓	✓	✓	✓		✓			✓	✓
Skip and repeat	✓	✓								
Data corruption								✓		✓
Incorrect instruction execution	✓	✓			✓	✓		✓		✓
Control flow error	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Variable corruption						✓				✓
Test inversion										

4.3. Countermeasures Based on Signature Computation

CIFER [54] is a CFI verification system based on RISC-V Instruction Trace Encoder and developed by A. Zgheib et al. It ensures the integrity of the program's control flow, which is crucial for preventing attacks such as code injection. The authors explain how their CFI verification system, based on the RISC-V trace encoder, enables the detection of instruction skip attacks on function calls, their returns, and branch instructions. Furthermore, it can detect more complex fault models, such as the corruption of any discontinuity instruction. A custom tool performs static analysis on the binary code, generating metadata that describe the control flow of the program. These metadata (stored in a dedicated RAM) include information about various types of discontinuity instructions such as calls, branches, and returns, as well as the associated memory indexes and the 32-bit program counter (PC).

The control flow monitoring is accomplished by tracing a sequence of addresses corresponding to the discontinuity instructions stored in the RAM. Each instruction requires a 96-bit memory space, comprising 32 bits for the instruction's address, 32 bits for the instruction itself, and two 16-bit fields for memory indexes. The system differentiates between unconditional and conditional branch instructions, storing appropriate memory indexes for each case. The authors use the trace encoder (TE), which is an instruction tracer that compresses, at runtime, the sequence of instructions executed by the RISC-V core into trace packets. They exploit the TE functionality by adding external blocks reading the TE packets in order to verify at runtime the program's CFI. A static analysis is carried out on the binary code where CFG metadata are generated. This information is stored in

a memory connected to the CFI verification module: the trace verifier (TV). At runtime, the TV receives TE packets and refers to the static data to check the CFI of the program. The TV is connected to the TE. Hence, the RISC-V core remains intact. CIFER does not require modifying the compiler, nor the ISA, nor the user code. Execution integrity is provided by comparing the precalculated signature with the computed signature of the micro-architecture control signals.

Werner et al. [56] also designed a protection for the conditional branches by using encoded comparisons. To detect faults in the control flow, the authors reused the concept of generalized path signature analysis (GPSA). The basic idea is to insert signature updates into the program code in such a way that independent of the used paths in a CFG; the signature value at a given instruction is always the same. The authors analyzed the functional requirements for derived signature calculation but also performed an evaluation of actual signature functions. Using a CRC with a suitable polynomial, any error in a single cycle and at least seven bit-flips, spread across two cycles, can be detected.

CIFER enhances security by comparing pre-calculated signatures with those computed at runtime from the microarchitecture control signals, ensuring the integrity of the program's control flow.

In comparison, Werner et al.'s approach to protecting conditional branches through generalized path signature analysis (GPSA) offers a different methodology by embedding signature updates within the program code. This technique ensures consistent signature values across different paths in the control flow graph (CFG), effectively detecting faults in control flow.

4.4. Countermeasures Based on IA

The increasing reliance on machine learning-based countermeasures for securing IoT systems against fault injection attacks is evident in recent research. These attacks, including voltage glitches, electromagnetic perturbations (EMFI), and clock glitches (CGFI), can compromise the integrity of embedded systems. Shrivastwa et al. [57] propose a two-stage ML-based detection framework to identify and classify such attacks. Their system, smart monitor, continuously analyzes sensor data from a chip being actively subjected to EMFI and CGFI attacks. However, the countermeasure does not explicitly state whether it is designed for cryptographic algorithms, processor architectures, or other security functions. Additionally, its cost-effectiveness and broader applicability to processor architectures remain unclear, as the study primarily focuses on its functionality within an experimental setup. High-level synthesis (HLS) validation confirms the system's effectiveness in detecting fault injections, but further research is needed to evaluate its real-world deployment feasibility and economic viability. Meanwhile, Asier Gambra et al. [58] presents an AI-driven approach to detecting voltage fault injection attacks by analyzing clock signal variations in embedded systems. The countermeasure targets STM32F410 microcontrollers, where power supply glitches can manipulate the clock, leading to faulty executions or system resets. Unlike traditional defenses, the proposed method uses a multi-layer perceptron (MLP) model to analyze clock traces in real time, distinguishing between normal execution, minor glitches, and severe faults. By correlating clock instability with power anomalies, the system enhances detection accuracy, offering a scalable and adaptive countermeasure for securing embedded devices. Mahmoud et al. [9] provides a systematic analysis of fault injection threats, emphasizing software–hardware hybrid defenses that integrate machine learning-based anomaly detection for real-time security. These countermeasures leverage AI-driven models to detect clock signal anomalies, power fluctuations, and execution faults, enhancing the resilience of embedded systems against voltage glitches, electromagnetic perturbations, and clock glitches. However, most of the surveyed countermeasures do

not explicitly target processor architectures. As a result, they fall outside the scope of our survey, which prioritizes processor-level fault injection defenses.

5. Discussion

5.1. Overheads and Tradeoffs

GPSA (Generalized path signature analysis) [56] exhibits low area overhead (4%), making it an attractive solution for hardware-constrained devices. However, this comes at the cost of a high execution time overhead (32%), as GPSA relies on the binary encoding of program instructions and CRC32 signature computation. While it does not require pipeline modifications, compiler modifications are necessary to integrate the signature verification mechanism. This significant runtime cost may limit its adoption in high-performance applications.

SOFIA [43] is primarily based on indirect branch elimination, which contributes to a relatively high area overhead (28.2%), but moderate execution time overhead (13.7%). Unlike GPSA, SOFIA does not require compiler modifications, simplifying its integration. However, pipeline modifications are necessary, which increases complexity and makes it less flexible compared to countermeasures that operate purely at the software level. The increased area overhead stems from the complexity of dispatchers, which restricts the number of instructions per basic block and increases the software overhead.

SCFP (sponge-based control-flow protection) [44] prioritizes low execution time overhead (9.1%), making it one of the most efficient runtime countermeasures. However, it introduces a high area overhead (28.8%), mainly due to the cryptographic operations used for integrity verification. Unlike SOFIA, SCFP requires compiler modifications but does not alter the pipeline, making it easier to integrate into existing processor architectures.

SCI-FI (control signal, code, and control flow integrity against fault injection attacks) [49] balances hardware and software costs, with moderate area overhead (6.5%) and higher execution time overhead (17.5%). Its overheads stem from the need to verify control-flow integrity at runtime, introducing additional signature computations. SCI-FI requires both compiler and pipeline modifications, making it less flexible than GPSA and SCFP.

CONFIDAENT [46] provides area and execution time overheads ranging from 0% to 36%, depending on the security level required. CONFIDAENT does not require compiler modifications, simplifying adoption. However, pipeline modifications are necessary, which can contribute to increased area overhead depending on implementation choices.

CIFER (code integrity and control flow verification for programs executed on a RISC-V core) [55] is a hardware-supported countermeasure that ensures both code integrity and control-flow verification for RISC-V architectures. It introduces a high area overhead (35.1% to 55%), making it one of the most resource-intensive solutions. However, unlike many other countermeasures, CIFER does not require compiler or pipeline modifications, making it easier to integrate without requiring significant software changes. CIFER operates at the hardware level, making it highly efficient in execution, but its large area cost makes it impractical for low-resource embedded systems.

CCFI-Cache (code and control-flow integrity cache) [52] is a transparent and flexible hardware-based protection mechanism that provides CFI and code integrity verification without requiring compiler or pipeline modifications. It introduces a moderate area overhead (10%) but exhibits a wide range of execution time overhead (2% to 63%), depending on system configurations and workload behavior. This flexibility makes CCFI-Cache well suited for systems where hardware modifications are preferable over software interventions. However, its worst-case execution time overhead (63%) is significantly higher than that of GPSA and SCFP, making it less predictable in terms of performance impact.

SecDec [53] is a hardware-based protection mechanism that ensures secure instruction decoding through masking with generated signals. It introduces an area overhead ranging from 3.71% to 16.93%, making it one of the least resource-intensive countermeasures. However, it requires compiler modifications but does not require pipeline modifications, simplifying integration while maintaining security. SecDec is particularly well suited for systems requiring lightweight security solutions with minimal impact on area overhead.

5.2. Portability to RISC-V

RISC-V's openness, modularity, and extensibility create a unique environment for exploring and implementing fault injection countermeasures. Researchers and designers can exploit its customizable architecture to integrate tailored security mechanisms at both the hardware and software levels. By leveraging these features, RISC-V can potentially achieve higher resilience to fault injection attacks compared to traditional closed-source processors. For this reason, we present in this section the countermeasures that have been effective with RISC-V. Additionally, we will discuss the applicability of other countermeasures to the RISC-V architecture.

The RISC-V CV32E40P core is the target of different countermeasure implementations. Werner et al. [44] utilized the CV32E40P in an ASIC implementation using UMC 65 nm technology. Savry et al. [46] and Thomas Chamelot [49] also implemented CONFIDAENT, SCI-FI on the CV32E40P, with Chamelot using 22nm technology. This core is favored for its open-source nature, extensibility, and support for custom security features.

Few works have chosen alternative targets. For instance, Danger et al. [52] implemented the CCFI-cache on an Artix-7 FPGA running the RISC-V PicoRV32. The PicoRV32 core differs from the CV32E40P in that it is a smaller, more lightweight core optimized for minimal resource usage on FPGA platforms.

SOFIA countermeasure focuses on enforcing security policies in hardware to protect against code reuse and code injection attacks. While the original implementation of SOFIA may have been demonstrated on specific processor architectures such as LEON3 or Xilinx Virtex-6, the core principles and concepts of SOFIA can be adapted and applied to other processor architectures, including RISC-V. However, it is worth noting that the adaptability and portability of the SOFIA architecture make it theoretically feasible to be applied to different processor architectures.

5.3. Real-World Scenarios

The countermeasure proposed by M. Werner et al. [56] targeting ARMv7 Cortex-M3 has seen partial real-world application, but its adoption is not widespread across commercial products. Similarly, the SOFIA [43] by R. de Clercq et al. has been applied in real-world settings, but its adoption remains limited to specific domains, primarily in aerospace and critical embedded systems. While RISC-V offers a promising platform for implementing fault injection countermeasures, the reality is that most existing implementations are research-driven rather than industry-adopted. The primary barriers to real-world adoption include the evolving nature of RISC-V security frameworks, the lack of industry-wide standardization, and the fact that many of these solutions are still in an experimental phase.

It is interesting to highlight that the described countermeasures present a non-negligible overhead in terms of resources, performance, or both, as shown in Table 4. This is largely due to the complexity and variety of errors that can be injected by attackers. Besides the expected increment of resource usage, it can be inferred that most solutions impact quite significantly the operating frequency of the design, or the latency of operations due to additional time required for verification. However, a complete and thorough

comparison of these solutions is not easy, as they have been prototyped on several different technologies (both ASICs and FPGAs).

As already pointed out, the physical security of microprocessors is continuously evolving, both from the point of view of newly discovered vulnerabilities or proposed countermeasures. In our opinion, it is essential that these threats are considered throughout the design flow, not only comprising the hardware design but also understanding the consequences that design choices have on the software stack (e.g., compiler choices and optimizations).

Table 4. Comparison of existing countermeasures based on area and execution time overheads, compiler and pipeline modifications, and target Devices.

Countermeasure	Overhead		Modifications		Target Device
	Area	Execution Time	Compiler	Pipeline	
SCFP [44]	28.8%	9.1%	✓	×	ASIC UMC 65 nm
SCI-FI [49]	6.5%	17.5%	✓	✓	ASIC FDSOI 22 nm
MAFIA CRC [51]	6.5%	18.4%	✓	✓	ASIC FDSOI 22 nm
MAFIA CBC-MAC [51]	23.8%	39%	✓	✓	ASIC FDSOI 22 nm
SOFIA [43]	28.2%	13.7%	×	✓	FPGA Virtex 6
CONFIDAENT [46]	0% to 36%	0% to 36%	×	✓	FPGA
CCFI-CACHE [52]	10%	2% to 63%	×	×	FPGA Artix 7
ATRIUM [47]	<20%	N/A	×	×	FPGA Virtex-7
SecDec [53]	3.71% to 16.93%	N/A	✓	×	GF22FDX FD-SOI 22 nm
GPSA [56]	4%	32%	✓	×	ASIC UMC 130 nm
HAPEI [45]	N/A	N/A	×	✓	N/A
CIFER [55]	35.1% to 55%	N/A	×	×	FPGA Artix 7

6. Conclusions

Fault injection attacks (FIAs) represent a critical threat due to their ability to induce erroneous behavior in hardware, potentially leading to severe security breaches. These attacks can take various forms, such as clock glitching, voltage glitching, electromagnetic injection, laser injection, and X-ray perturbation. Each method has its own strengths and weaknesses in terms of precision, cost, and complexity, highlighting the need for tailored countermeasures. This paper has explored the various ways processors can be attacked via fault injection and reviewed the existing methods to protect them, with a particular focus on the RISC-V architecture.

Our review reveals that current countermeasures against FIAs are highly dependent on the attacker's goals, such as altering execution flow or corrupting data. These countermeasures range from securing cryptographic algorithms and protecting against reverse engineering to implementing specialized security measures at the design stage. For instance, duplication and triplication of critical operations can mitigate some forms of fault injection but may fall short against sophisticated attacks like multi-instruction skips induced by electromagnetic pulses or laser injections.

Comparing these security measures highlights the important features that good protection systems should have. Some of these countermeasures have been specifically applied to the RISC-V processor, while others can be adapted for use with RISC-V. While physical access remains a limiting factor for many FIAs, the evolving landscape of hardware security demands continuous advancement in countermeasure techniques to protect against increasingly sophisticated attacks.

In conclusion, the insights and comparisons provided in this paper aim to aid in the development of new, more effective countermeasures against fault injection attacks, ensuring the integrity and security of digital systems.

Funding: This work is partially supported by the ARSENE project funded by the "France 2030" government investment plan managed by the French National Research Agency (ANR), under the reference ANR-22-PECY-0004 (https://www.pepr-cybersecurite.fr/projet/arsene/, accessed on 13 March 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Amiel, F.; Villegas, K.; Feix, B.; Marcel, L. Passive and Active Combined Attacks: Combining Fault Attacks and Side Channel Analysis. In Proceedings of the Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC 2007), Vienna, Austria, 10 September 2007; pp. 92–102. [CrossRef]
2. Kocher, P.C.; Jaffe, J.; Jun, B. Differential Power Analysis. In *Advances in Cryptology—CRYPTO '99, Proceedings of the 19th Annual International Cryptology Conference, Santa Barbara, CA, USA, 15–19 August 1999, Proceedings*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 1999; Volume 1666, pp. 388–397. [CrossRef]
3. Brier, E.; Clavier, C.; Olivier, F. Correlation Power Analysis with a Leakage Model. In *Cryptographic Hardware and Embedded Systems-CHES 2004, Proceedings of the 6th International Workshop, Cambridge, MA, USA, 11–13 August 2004, Proceedings 6*; Joye, M., Quisquater, J.J., Eds.; Springer: Berlin/Heidelberg, Germany, 2004; pp. 16–29.
4. Boneh, D.; DeMillo, R.A.; Lipton, R.J. On the Importance of Checking Cryptographic Protocols for Faults. In *Advances in Cryptology—EUROCRYPT'97, Proceedings of the International Conference on the Theory and Application of Cryptographic Techniques Konstanz, Germany, 11–15 May 1997, Proceedings*; Fumy, W., Ed.; Springer: Berlin/Heidelberg, Germany, 1997; pp. 37–51. [CrossRef]
5. Yuce, B.; Schaumont, P.; Witteman, M. Fault Attacks on Secure Embedded Software: Threats, Design, and Evaluation. *J. Hardw. Syst. Secur.* **2018**, *2*, 111–130. [CrossRef]
6. Akdemir, K.D.; Wang, Z.; Karpovsky, M.; Sunar, B. Design of Cryptographic Devices Resilient to Fault Injection Attacks Using Nonlinear Robust Codes. In *Fault Analysis in Cryptography*; Joye, M., Tunstall, M., Eds.; Springer: Berlin/Heidelberg, Germany, 2012; pp. 171–199. [CrossRef]
7. Mishra, J.; Sahay, S.K. Modern Hardware Security: A Review of Attacks and Countermeasures. *arXiv* **2025**, arXiv:2501.04394. Available online: http://arxiv.org/abs/2501.04394 (accessed on 13 March 2025).
8. Shuvo, A.M.; Zhang, T.; Farahmandi, F.; Tehranipoor, M. A comprehensive survey on non-invasive fault injection attacks. *Cryptol. ePrint Arch.* **2023** .
9. Gangolli, A.; Mahmoud, Q.H.; Azim, A. A systematic review of fault injection attacks on IOT systems. *Electronics* **2022**, *11*, 2023. [CrossRef]
10. Kazemi, Z.; Hely, D.; Fazeli, M.; Beroulle, V. A Review on Evaluation and Configuration of Fault Injection Attack Instruments to Design Attack Resistant MCU-Based IoT Applications. *Electronics* **2020**, *9*, 1153. [CrossRef]
11. Barenghi, A.; Breveglieri, L.; Koren, I.; Naccache, D. Fault Injection Attacks on Cryptographic Devices: Theory, Practice, and Countermeasures. *Proc. IEEE* **2012**, *100*, 3056–3076. [CrossRef]
12. Canella, C.; Van Bulck, J.; Schwarz, M.; Lipp, M.; Von Berg, B.; Ortner, P.; Piessens, F.; Evtyushkin, D.; Gruss, D. A systematic evaluation of transient execution attacks and defenses. In Proceedings of the 28th USENIX Security Symposium (USENIX Security 19), Santa Clara, CA, USA, 14–16 August 2019; pp. 249–266.
13. Ge, Q.; Yarom, Y.; Cock, D.; Heiser, G. A survey of microarchitectural timing attacks and countermeasures on contemporary hardware. *J. Cryptogr. Eng.* **2018**, *8*, 1–27. [CrossRef]
14. Xiong, W.; Szefer, J. Survey of transient execution attacks and their mitigations. *ACM Comput. Surv. (CSUR)* **2021**, *54*, 1–36. [CrossRef]

15. Agoyan, M.; Dutertre, J.M.; Naccache, D.; Robisson, B.; Tria, A. When Clocks Fail: On Critical Paths and Clock Faults. In *Smart Card Research and Advanced Application, Proceedings of the 9th IFIP WG 8.8/11.2 International Conference, CARDIS 2010, Passau, Germany, 14–16 April 2010, Proceedings*; Springer: Berlin/Heidelberg, Germany, 2010; Volume 6035, pp. 182–193. [CrossRef]
16. Claudepierre, L.; Péneau, P.Y.; Hardy, D.; Rohou, E. TRAITOR: A Low-Cost Evaluation Platform for Multifault Injection. In Proceedings of the 2021 International Symposium on Advanced Security on Software and Systems, Virtual Event, Hong Kong, 7 June 2021; pp. 51–56. [CrossRef]
17. NewAE Technology Inc. ChipWhisperer Documentation. Online Resource. Available online: https://chipwhisperer.readthedocs.io/en/latest/ (accessed on 13 March 2025).
18. Korak, T.; Hoefler, M. On the Effects of Clock and Power Supply Tampering on Two Microcontroller Platforms. In Proceedings of the 2014 Workshop on Fault Diagnosis and Tolerance in Cryptography, Busan, Republic of Korea, 23 September 2014; pp. 8–17. [CrossRef]
19. Chen, Z.; Vasilakis, G.; Murdock, K.; Dean, E.; Oswald, D.; Garcia, F.D. VoltPillager: Hardware-Based Fault Injection Attacks Against Intel SGX Enclaves Using the SVID Voltage Scaling Interface. In Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), Vancouver, BC, Canada, 11–13 August 2021; pp. 699–716.
20. Buhren, R.; Jacob, H.N.; Krachenfels, T.; Seifert, J.P. One Glitch to Rule Them All: Fault Injection Attacks Against AMD's Secure Encrypted Virtualization. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, 15–19 November 2021; pp. 2875–2889. [CrossRef]
21. Dumont, M.; Lisart, M.; Maurine, P. Modeling and Simulating Electromagnetic Fault Injection. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **2021**, *40*, 680–693. [CrossRef]
22. Ordas, S.; Guillaume-Sage, L.; Maurine, P. Electromagnetic Fault Injection: The Curse of Flip-Flops. *J. Cryptogr. Eng.* **2017**, *7*, 183–197. [CrossRef]
23. Trouchkine, T.; Bouffard, G.; Clédière, J. Fault Injection Characterization on Modern CPUs: From the ISA to the Micro-Architecture. In Proceedings of the 13th IFIP International Conference on Information Security Theory and Practice (WISTP), Paris, France, 11–12 December 2019; pp. 123–138 . [CrossRef]
24. Anceau, S.; Bleuet, P.; Clédière, J.; Maingault, L.; Rainard, J.L.; Tucoulou, R. Nanofocused X-Ray Beam to Reprogram Secure Circuits. In *Cryptographic Hardware and Embedded Systems—CHES 2017, Proceedings of the 19th International Conference, Taipei, Taiwan, 25–28 September 2017, Proceedings*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2017; Volume 10529, pp. 175–188. [CrossRef]
25. Kim, Y.; Daly, R.; Kim, J.; Fallin, C.; Lee, J.H.; Lee, D.; Wilkerson, C.; Lai, K.; Mutlu, O. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. In Proceedings of the 41st Annual International Symposium on Computer Architecuture, ISCA '14, Minneapolis, MN, USA, 14–18 June 2014; pp. 361–372.
26. Murdock, K.; Oswald, D.; Garcia, F.D.; Van Bulck, J.; Gruss, D.; Piessens, F. Plundervolt: Software-based Fault Injection Attacks against Intel SGX. In Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 18–21 May 2020; pp. 1466–1482. [CrossRef]
27. Gonidec, G.L.; Real, M.M.; Bouffard, G.; Prévotet, J.C. Do Not Trust Power Management: A Survey on Internal Energy-based Attacks Circumventing Trusted Execution Environments Security Properties. *arXiv* **2024**, arXiv:2405.15537. Available online: http://arxiv.org/abs/2405.15537 (accessed on 13 March 2025).
28. Trouchkine, T.; Bukasa, S.K.; Escouteloup, M.; Lashermes, R.; Bouffard, G. Electromagnetic Fault Injection Against a System-on-Chip, Toward New Micro-Architectural Fault Models. *arXiv* **2019**, arXiv:1910.11566.
29. Trouchkine, T.; Bouffard, G.; Clédière, J. EM Fault Model Characterization on SoCs: From Different Architectures to the Same Fault Model. In Proceedings of the 2021 Workshop on Fault Detection and Tolerance in Cryptography (FDTC), Milan, Italy, 17 September 2021; pp. 31–38. [CrossRef]
30. Laurent, J.; Deleuze, C.; Pebay-Peyroula, F.; Beroulle, V. Bridging the Gap between RTL and Software Fault Injection. *J. Emerg. Technol. Comput. Syst.* **2021**, *17*, 24. [CrossRef]
31. Yuce, B.; Ghalaty, N.F.; Santapuri, H.; Deshpande, C.; Patrick, C.; Schaumont, P. Software Fault Resistance is Futile: Effective Single-Glitch Attacks. In Proceedings of the 2016 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC), Santa Barbara, CA, USA, 16 August 2016; pp. 47–58. [CrossRef]
32. Alshaer, I.; Colombier, B.; Deleuze, C.; Beroulle, V.; Maistri, P. Microarchitecture-Aware Fault Models: Experimental Evidence and Cross-Layer Inference Methodology. In Proceedings of the 2021 16th International Conference on Design & Technology of Integrated Systems in Nanoscale Era (DTIS), Montpellier, France, 28–30 June 2021; pp. 1–6. [CrossRef]
33. Werner, V.; Maingault, L.; Potet, M.L. An End-to-End Approach for Multi-Fault Attack Vulnerability Assessment. In Proceedings of the 2020 Workshop on Fault Detection and Tolerance in Cryptography (FDTC), Milan, Italy, 13 September 2020; pp. 10–17. [CrossRef]
34. Elmohr, M.A.; Liao, H.; Gebotys, C.H. EM Fault Injection on ARM and RISC-V. In Proceedings of the 2020 21st International Symposium on Quality Electronic Design (ISQED), Santa Clara, CA, USA, 25–26 March 2020; pp. 206–212.

35. Amin, K.; Gebotys, C.; Faraj, M.; Liao, H. Analysis of Dynamic Laser Injection and Quiescent Photon Emissions on an Embedded Processor. *J. Hardw. Syst. Secur.* **2020**, *4*, 55–67. [CrossRef]
36. Breier, J.; Jap, D.; Chen, C.N. Laser Profiling for the Back-Side Fault Attacks: With a Practical Laser Skip Instruction Attack on AES. In Proceedings of the 1st ACM Workshop on Cyber-Physical System Security, Singapore, 4 April 2015; pp. 99–103.
37. Colombier, B.; Menu, A.; Dutertre, J.M.; Moëllic, P.A.; Rigaud, J.B.; Danger, J.L. Laser-Induced Single-Bit Faults in Flash Memory: Instructions Corruption on a 32-Bit Microcontroller. In Proceedings of the 2019 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), McLean, VA, USA, 5–10 May 2019; pp. 1–10. [CrossRef]
38. Timmers, N.; Spruyt, A.; Witteman, M. Controlling PC on ARM Using Fault Injection. In Proceedings of the 2016 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC), Santa Barbara, CA, USA, 16 August 2016; pp. 25–35. [CrossRef]
39. Alshaer, I.; Colombier, B.; Deleuze, C.; Beroulle, V.; Maistri, P. Variable-Length Instruction Set: Feature or Bug? In Proceedings of the 2022 25th Euromicro Conference on Digital System Design (DSD), Maspalomas, Spain, 31 August–2 September 2022; pp. 464–471. [CrossRef]
40. Alshaer, I.; Colombier, B.; Deleuze, C.; Beroulle, V.; Maistri, P. Microarchitectural Insights into Unexplained Behaviors Under Clock Glitch Fault Injection. In *Smart Card Research and Advanced Applications*; Lecture Notes in Computer Science; Bhasin, S.; Roche, T., Eds.; Springer: Berlin/Heidelberg, Germany, 2024; Volume 14530.
41. Alshaer, I.; Al-kaf, A.; Egloff, V.; Beroulle, V. Inferred Fault Models for RISC-V and Arm: A Comparative Study. In Proceedings of the 2024 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), Didcot, UK, 8–10 October 2024; pp. 1–6.
42. Di Natale, G.; Gizopoulos, D.; Di Carlo, S.; Bosio, A.; Canal, R. *Cross-Layer Reliability of Computing Systems*; IET—The Institution of Engineering and Technology: London, UK, 2020. [CrossRef]
43. Clercq, R.; Keulenaer, R.; Coppens, B.; Yang, B.; Maene, P.; De Bosschere, K.; Preneel, B.; De Sutter, B.; Verbauwhede, I. SOFIA: Software and Control Flow Integrity Architecture. In Proceedings of the 2016 International Conference on Field Programmable Logic and Applications, Dresden, Germany, 14–18 March 2016; pp. 1172–1177. [CrossRef]
44. Werner, M.; Unterluggauer, T.; Schaffenrath, D.; Mangard, S. Sponge-Based Control-Flow Protection for IoT Devices. In Proceedings of the 2018 IEEE European Symposium on Security and Privacy (EuroS&P), London, UK, 24–26 April 2018; pp. 214–226. [CrossRef]
45. Lashermes, R.; Bouder, H.; Thomas, G. Secure IT Systems: Hardware-Assisted Program Execution Integrity: HAPEI. In *Secure IT Systems, Proceedings of the 23rd Nordic Conference, NordSec 2018, Oslo, Norway, 28–30 November 2018, Proceedings*; Lecture Notes in Computer Science; Springer International Publishing: Cham, Switzerland, 2018; Volume 11252. [CrossRef]
46. Savry, O.; El-Majihi, M.; Hiscock, T. Confidaent: Control FLow Protection with Instruction and Data Authenticated Encryption. In Proceedings of the 2020 23rd Euromicro Conference on Digital System Design (DSD), Kranj, Slovenia, 26–28 August 2020; pp. 246–253. [CrossRef]
47. Zeitouni, S.; Dessouky, G.; Arias, O.; Sullivan, D.; Ibrahim, A.; Jin, Y.; Sadeghi, A.R. ATRIUM: Runtime Attestation Resilient Under Memory Attacks. In Proceedings of the 2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Irvine, CA, USA, 13–16 November 2017; pp. 384–391. [CrossRef]
48. Nasahl, P.; Sultana, S.; Liljestrand, H.; Grewal, K.; LeMay, M.; Durham, D.M.; Schrammel, D.; Mangard, S. EC-CFI: Control-Flow Integrity via Code Encryption Counteracting Fault Attacks. In Proceedings of the 2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), San Jose, CA, USA, 1–4 May 2023; pp. 24–35. [CrossRef]
49. Chamelot, T.; Couroussé, D.; Heydemann, K. SCI-FI: Control Signal, Code, and Control Flow Integrity against Fault Injection Attacks. In Proceedings of the 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE), Antwerp, Belgium, 14–23 March 2022; pp. 556–559. [CrossRef]
50. Wilken, K.; Shen, J. Continuous Signature Monitoring: Efficient Concurrent Detection of Processor Control Errors. In Proceedings of the International Test Conference 1988 Proceedings: New Frontiers in Testing, Washington, DC, USA, 12–14 September 1988; pp. 914–925. [CrossRef]
51. Chamelot, T.; Couroussé, D.; Heydemann, K. MAFIA: Protecting the Microarchitecture of Embedded Systems Against Fault Injection Attacks. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **2023**, *42*, 4555–4568. [CrossRef]
52. Danger, J.L.; Facon, A.; Guilley, S.; Heydemann, K.; Kühne, U.; Si Merabet, A.; Timbert, M. CCFI-Cache: A Transparent and Flexible Hardware Protection for Code and Control-Flow Integrity. In Proceedings of the 2018 21st Euromicro Conference on Digital System Design (DSD), Prague, Czech Republic, 29–31 August 2018; pp. 529–536. [CrossRef]
53. Leplus, G.; Savry, O.; Bossuet, L. SecDec: Secure Decode Stage Thanks to Masking of Instructions with the Generated Signals. In Proceedings of the 2022 25th Euromicro Conference on Digital System Design (DSD), Maspalomas, Spain, 31 August–2 September 2022; pp. 556–563. [CrossRef]
54. Zgheib, A.; Potin, O.; Rigaud, J.B.; Dutertre, J.M. A CFI Verification System Based on the RISC-V Instruction Trace Encoder. In Proceedings of the 2022 25th Euromicro Conference on Digital System Design (DSD), Maspalomas, Spain, 31 August–2 September 2022; pp. 456–463. [CrossRef]

55. Zgheib, A.; Potin, O.; Rigaud, J.B.; Dutertre, J.M. CIFER: Code Integrity and Control Flow Verification for Programs Executed on a RISC-V Core. In Proceedings of the 2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), San Jose, CA, USA, 1–4 May 2023; pp. 100–110. [CrossRef]
56. Werner, M.; Wenger, E.; Mangard, S. Protecting the control flow of embedded processors against fault attacks. In *Smart Card Research and Advanced Applications, Proceedings of the 14th International Conference, CARDIS 2015, Bochum, Germany, 4–6 November 2015. Revised Selected Papers 14*; Springer: Berlin/Heidelberg, Germany, 2016; pp. 161–176.
57. Shrivastwa, R.R.; Guilley, S.; Danger, J.L. Multi-source fault injection detection using machine learning and sensor fusion. In *Security and Privacy, Proceedings of the Second International Conference, ICSP 2021, Jamshedpur, India, 16–17 November 2021, Proceedings*; Springer: Berlin/Heidelberg, Germany, 2021; pp. 93–107.
58. Gambra, A.; Chatterjee, D.; Rioja, U.; Armendariz, I.; Batina, L. Machine Learning-Based Detection of Glitch Attacks in Clock Signal Data. *Cryptol. ePrint Arch.* **2024**.